Summary

This RFC describes the backend & data model for BJJ Tracker MVP using Node.js + Express and PostgreSQL. It covers database tables, relationships, API routes, validations, and constraints.

Goals

- Enable users to **record**, **categorize**, and **review** Brazilian Jiu-Jitsu techniques.
- Support a flexible organization using **tags** (type of move), **body position**, and **limb placement**.
- Allow users to attach **media** (images/videos) and **explanations** to each move.
- Provide **training sessions** and **flashcard review** for spaced repetition.

High-level architecture

- Frontend: React (Vite) + Tailwind
- Backend: Node.js + Express (TypeScript)
- Auth: JWT (access token short-lived + refresh token)
- DB: PostgreSQL (single primary DB for MVP)
- Media Storage: Cloud (AWS S3 or Firebase Storage)/ maybe just link to media online
- Hosting: Render / Vercel (Frontend), Railway / Render (Backend)

Core Entities & Relationships (ER overview)

Relationship	Type	Description
users → moves	**1–many**	Each user owns multiple moves
users → tags	**1–many**	Each user defines their own tags
users → sessions	**1–many**	Each user logs their own sessions
moves ↔ tags	**many–many**	Each move can have multiple tags
sessions ↔ moves	**many–many**	Each session can include multiple moves
moves ↔ flashcards	**1–1**	Each move can have only one flashcard

Core Entities

1. **Users**

Each user can create moves, organize them into sessions, and tag them.

Field	Type	Notes
id	SERIAL PRIMARY KEY	unique identifier
username	VARCHAR(50)	required
email	VARCHAR(100)	unique, required
password_hash	TEXT	stored securely
created_at	TIMESTAMP DEFAULT now()	creation timestamp

Relationships:

- One user → many moves
- One user → many tags
- One user → many sessions
- One user → many flashcards

2. **Moves**

   Main table representing BJJ moves/techniques.

Field	Type	Notes
id	SERIAL PRIMARY KEY	unique identifier
user_id	INTEGER REFERENCES users(id)	owner of the move
title	VARCHAR(255)	name of the move
body_position	VARCHAR(100)	e.g., Guard, Mount, Side Control
limb_placement	TEXT	description of grips and limb placement
explanation	TEXT	technical explanation or notes
media_url	TEXT	optional image/video link

```
created_at      | TIMESTAMP DEFAULT now()         | creation timestamp
updated_at      | TIMESTAMP DEFAULT now()       | last update timestamp
```

Relationships:

- One user → many moves
- Many moves ↔ many tags (via `move_tags`)
- Many moves ↔ many sessions (via `session_moves`)
- One move → one flashcard

3. **Tags**

List of move categories (type of move).

```
Field           | Type                      | Notes
----------------+---------------------------+-----------------------------------
-----
id              | SERIAL PRIMARY KEY         | unique identifier
name            | VARCHAR(50)                | e.g., Submission, Sweep, Escape
```

Relationships:

4. One user → many tags
5. Many tags ↔ many moves (via `move_tags`)

6. **move_tags**

   Join table for many-to-many relationship between moves and tags.

```
Field           | Type                            | Notes
----------------+---------------------------------+-----------------------
-----------------
```

```
move_id         | INTEGER REFERENCES moves(id)   | move reference
tag_id          | INTEGER REFERENCES tags(id)    | tag reference
PRIMARY KEY     | (move_id, tag_id)              | ensures uniqueness
```

Because tags are user-scoped, both `move_id` and `tag_id` will always belong to the same user, maintaining data isolation.

7. sessions

Training log for tracking practice.

```
Field           | Type                           | Notes
----------------+--------------------------------+-----------------------------
-----------
id              | SERIAL PRIMARY KEY             | unique identifier
user_id         | INTEGER REFERENCES users(id)   | owner
title           | VARCHAR(255)                   | session title
notes           | TEXT                           | reflections or learnings
created_at      | TIMESTAMP DEFAULT now()        | timestamp
```

Relationships:

- One user → many sessions
- Many sessions ↔ many moves (via `session_moves`)

8. session_moves

Join table for moves practiced in a session.

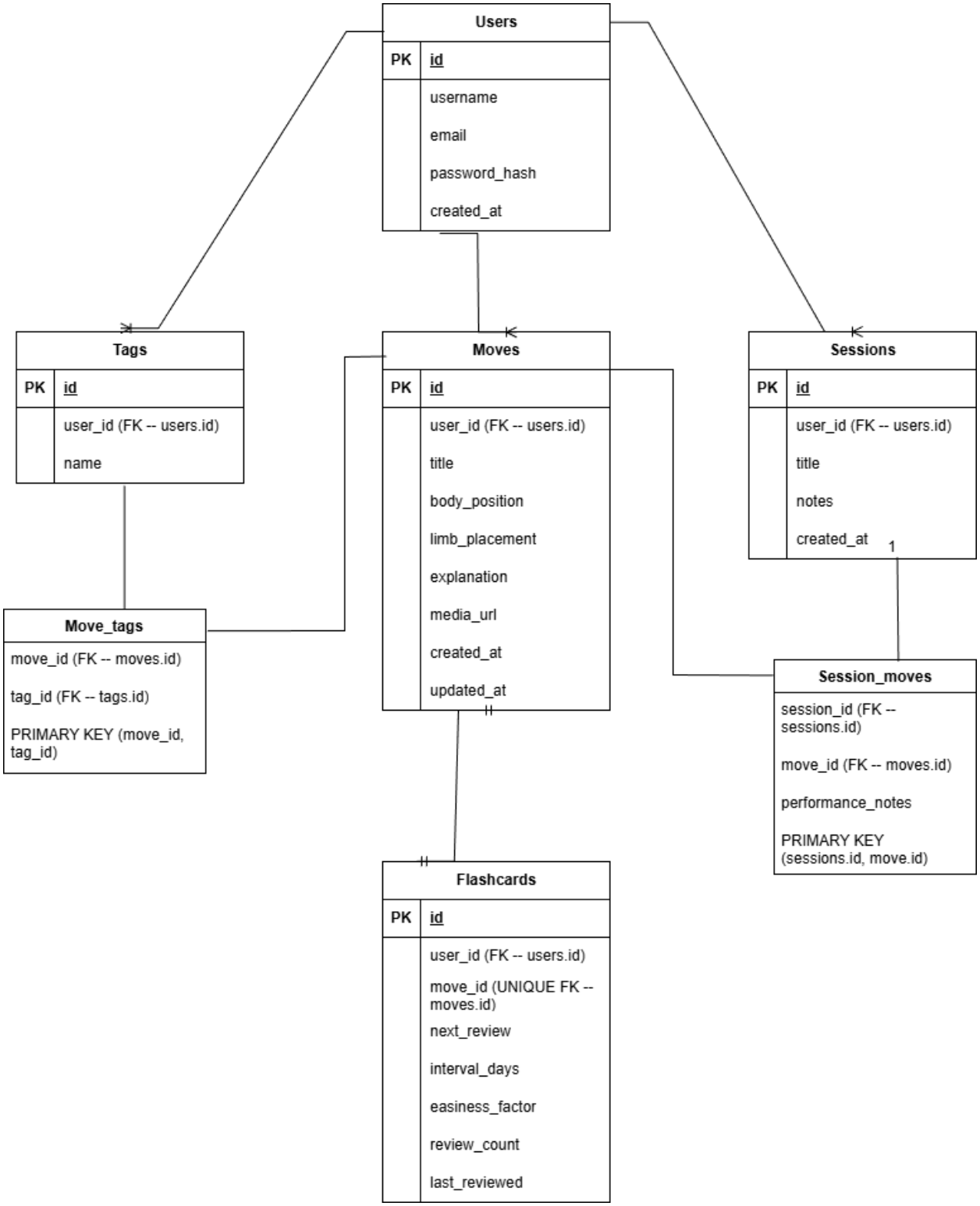
```
Field               | Type                              | Notes
--------------------+-----------------------------------+----------------------
------------------
session_id          | INTEGER REFERENCES sessions(id)   | session reference
move_id             | INTEGER REFERENCES moves(id)      | practiced move
performance_notes   | TEXT                          | reflections or feedback
PRIMARY KEY         | (session_id, move_id)             | ensures uniqueness
```

Each entry always ties to one user's session and one of *their own* moves.

9. Flashcards

Table for flashcard-based learning and spaced repetition review of moves.

Field	Type	Notes
id	SERIAL PRIMARY KEY	unique identifier
user_id	INTEGER REFERENCES users(id)	owner
move_id	INTEGER REFERENCES moves(id)	one-to-one link to move
next_review	TIMESTAMP	next time the flashcard should appear
interval_days	INTEGER	spaced repetition intervals
easiness_factor	FLOAT DEFAULT 2.5	SM-2 algorithm factor
review_count	INTEGER DEFAULT 0	number of reviews
last_reviewed	TIMESTAMP	last review timestamp


Users
PK id
username
email
password_hash
created_at
Tags
PK id
user_id (FK -- users.id)
name
Moves
PK id
user_id (FK -- users.id)
title
body_position
limb_placement
explanation
media_url
created_at
updated_at
Sessions
PK id
user_id (FK -- users.id)
title
notes
created_at
1
Move_tags
move_id (FK -- moves.id)
tag_id (FK -- tags.id)
PRIMARY KEY (move_id, tag_id)
Session_moves
session_id (FK -- sessions.id)
move_id (FK -- moves.id)
performance_notes
PRIMARY KEY (sessions.id, move.id)
Flashcards
PK id
user_id (FK -- users.id)
move_id (UNIQUE FK -- moves.id)
next_review
interval_days
easiness_factor
review_count
last_reviewed

Data Flow

1. User logs in → receives JWT.
2. User creates a move → stored in `moves`.
3. User tags it → entries added to `move_tags`.
4. User creates a training session → logs practiced moves via `session_moves`.
5. User reviews moves via flashcards → performance updates next review interval.

API Endpoints

Move Management

POST `/api/moves` → Create a new move

Request Body (JSON):

```
{
  "title": "Armbar from Guard",
  "body_position": "Guard",
  "limb_placement": "Right leg across opponent's torso, left leg over
the opponent's head.",
  "explanation": "Control the opponent's arm, pivot your hips, and
apply pressure to the elbow joint.",
  "media_url": "https://cdn.example.com/armbar.mp4",
  "tags": ["Submission", "From Guard"]
}
```

Backend logic would:

1. Create a new record in `moves`.
2. For each tag name:
    - Find or create the tag in `tags`.
    - Create a record in `move_tags (move_id, tag_id)`.

Example Response (201 Created):


```
{
  "id": 17,
  "user_id": 5,
  "title": "Armbar from Guard",
  "body_position": "Guard",
  "limb_placement": "Right leg across opponent’s torso, left leg over the opponent’s head.",
  "explanation": "Control the opponent’s arm, pivot your hips, and apply pressure to the elbow joint.",
  "media_url": "https://cdn.example.com/armbar.mp4",
  "tags": ["Submission", "From Guard"],
  "created_at": "2025-11-12T16:45:00.000Z",
  "updated_at": "2025-11-12T16:45:00.000Z"
}
```


GET `/api/moves` → Get all moves for the current user


```
[
  {
    "id": 17,
    "user_id": 5,
    "title": "Armbar from Guard",
    "body_position": "Guard",
```

"limb_placement": "Right leg across opponent’s torso, left leg over the opponent’s head.",

"explanation": "Control the opponent’s arm, pivot your hips, and apply pressure to the elbow joint.",

"media_url": "https://cdn.example.com/armbar.mp4",

"created_at": "2025-11-12T16:45:00.000Z",

"updated_at": "2025-11-12T16:45:00.000Z"

},

{

"id": 18,

"user_id": 5,

"title": "Triangle Choke",

"body_position": "Guard",

"limb_placement": "Legs around the opponent’s neck and arm trapped.",

"explanation": "Use your legs to form a triangle around the opponent’s neck and pull their head down.",

"media_url": "https://cdn.example.com/triangle.mp4",

"created_at": "2025-11-12T17:00:00.000Z",

"updated_at": "2025-11-12T17:00:00.000Z"

}

]

GET `/api/moves/:id` → Get a specific move

{

"id": 17,

```
  "user_id": 5,

  "title": "Armbar from Guard",

  "body_position": "Guard",

  "limb_placement": "Right leg across opponent’s torso, left leg over the opponent’s head.",

  "explanation": "Control the opponent’s arm, pivot your hips, and apply pressure to the elbow joint.",

  "media_url": "https://cdn.example.com/armbar.mp4",

  "created_at": "2025-11-12T16:45:00.000Z",

  "updated_at": "2025-11-12T16:45:00.000Z"

}
```


NEED TO WORK ON THE IMPLEMENTATION OF FILTERING AND FETCHING DATA BY TAGS

PUT `/api/moves/:id` → Update a move

Request Body (JSON):


```
{

  "title": "Armbar from Closed Guard",

  "body_position": "Closed Guard",

  "limb_placement": "Right leg over opponent’s torso, left leg under neck.",

  "explanation": "Pivot hips to align with the trapped arm, squeeze knees together.",

  "media_url": "https://cdn.example.com/armbar-updated.mp4"

}
```

Response Example (200 OK):


```
{
  "id": 17,
  "user_id": 5,
  "title": "Armbar from Closed Guard",
  "body_position": "Closed Guard",
  "limb_placement": "Right leg over opponent’s torso, left leg under neck.",
  "explanation": "Pivot hips to align with the trapped arm, squeeze knees together.",
  "media_url": "https://cdn.example.com/armbar-updated.mp4",
  "created_at": "2025-11-12T16:45:00.000Z",
  "updated_at": "2025-11-12T18:10:00.000Z"
}
```


DELETE `/api/moves/:id` → Delete a move


```
{
  "message": "Move deleted successfully.",
  "deleted_id": 17
}
```


Tags

POST `/api/tags` → Create new tag
GET `/api/tags` → Get all tags

Delete and put as well

Sessions

POST `/api/sessions` → Create new training session
GET `/api/sessions` → Get all sessions
GET `/api/sessions/:id` → Get details for one session
POST `/api/sessions/:id/moves` → Attach moves to a session

Flashcards

GET `/api/flashcards` → Retrieve flashcards due for review
POST `/api/flashcards/review` → Submit review result (updates next_review, interval, etc.)
POST `/api/flashcards/create` → Generate flashcards for new moves
GET `/api/flashcards/:id` → View single flashcard details

Security

- JWT Authentication for all user-protected routes.
- Passwords stored using bcrypt hash.
- Role-based access (only owner can modify their data).